

Feature: Confirmations

From build-plan: feature 6

Branch: feature/06-confirmations

Status: verified. Whole feature seen and agreed by Frank 2026-10-02; steps 6.1 to 6.6 built, tested and reviewed step by step; no P0 or P1 was ever open; decisions 6 to 11 made with Frank along the way. The checkpoint for the final review.

Goal

When a booking is saved, the customer gets a confirmation email from the business, with the time in the business's own zone and a calendar invite (`.ics`) attached, and the business gets a notification it can read in a second. Customers only ever hear from the business: never from the product's name and never from a worker's own address. The same email path delivers the login codes, which today print to the console and stop the API from running in production.

Texts (the confirmation text and the reminder the evening before) are feature 8, with its job runner. Until then a customer who gave only a phone (5d's decision 16) gets no confirmation; the business notification still names them.

In scope

- One email seam in the backend: every send goes through Resend with the key it is given; in development without a key, one console line per email with ids only. Every send carries an idempotency key so a repeat never makes a second email.
- Each business sends from its own Resend account (decision 6): the app holds that account's send-only key, write-only and locked. Two ways in: Frank's client setup form for a new client he sets up, and the owner's own "Email sending" card after signing in, for a business that already has its email. Saving a key sends a test email first, so a wrong key is never stored.
- Login codes sent through it. Production refuses to start without the email settings, the way it already refuses without `CALENDAR_TOKEN_KEY`.
- The business details an email needs (decision 8): who it comes from, where the business notification goes, the phone for the `tel:` button, the website, the brand colour; the logo already has a column. Set when the agency sets up a client; editable in Settings later (feature 12).
- The calendar invite: a pure function that writes one booking as an RFC 5545 `.ics`.
- Two emails, in Primo's template pattern (`primo-painters/emails/contact-lead.tsx` : hex colours, never CSS variables; the zone pinned; a `tel:` button): the customer's confirmation and the business notification.

- Sending both once a booking is saved, without the customer's answer waiting for them (the same rule as the Google event, 5d's decision 6), and an `email_sent` timeline entry for each email that went.

Decisions made in the spec

1. **Emails are sent after the booking is saved and never make the customer wait** (the same rule as 5d's decision 6, Frank's F-93 answer). A failure keeps the booking; feature 8 retries. 2. **The same email is never sent twice.** Each send carries an idempotency key made from the booking id and the email's kind, and only a new booking sends (a resent form gets its booking back and nothing else). 3. **The timeline records that an email went, never what it said.** No address and no content in the payload or the logs. 4. **No sender, no email.** A business without its sender address, notification address or key sends nothing and logs one line, rather than sending from a fallback address the customer would not recognise. 5. **The invite's times are in UTC.** Every calendar converts them to the reader's own zone, so no time zone block is written; the email's text shows the business's zone. Rejected: a `VTIMEZONE` block, long and easy to get wrong, adding nothing when the times are absolute. 6. **Each business sends through its own Resend account, and nobody can read its key** (Frank, 2026-10-02, open question 1). The account is the business's, the one its Gmail replies already go out through (the agency's email setup guide); its emails and their logs stay there. The app holds a key made with Resend's "Sending access" only, limited to the business's domain: it can send, it cannot read past emails. The key is write-only: stored locked with the token key that locks calendar tokens, never returned by any route, shown on any screen or written to any log; changing it means pasting a new one. It lives in a table of its own, not on `organization`, which Better Auth's own routes return to members. A new client: Frank sets up the account, pastes the key in client setup, tests, and hands over the password (the client changes it and removes him). A business that already has its email: Frank builds with a test sender of his own, and at handover the owner pastes their own key in the "Email sending" card, replacing his. Rejected: one agency account sending for everyone, where Frank could read every business's emails in its logs. 7. **Login codes come from the agency** (Frank, 2026-10-02, open question 3): one address on the agency's own domain, through the agency's own Resend account (`RESEND_API_KEY`), named in the setting `LOGIN_EMAIL_FROM` . The app sends them, not the business, and a new owner needs a code before they can sign in and paste their own key. Rejected: each business's own sender, which cannot send the first code. 8. **One email per business now; each person's own work email later** (Frank, 2026-10-02, open question 2). The business notification goes to the business's notification address, which client setup fills with the owner's email to start; it is its own field so a business can point it at another inbox without the owner changing their sign-in. Who hears about a booking is answered in one place (`findBookingEmailRecipients` , 6.6), so when a person has their own work email at the business's domain (stored with Settings, feature 12), Jane's replies and that person's notification go to them without a rewrite. A work address at the business's domain is still the business; only a worker's personal address is ruled out. When a business runs on one email, the booked worker is kept in the loop by text (feature 8). Both later parts are written into the build plan. 9. **The owner's card also sets the two addresses** (Frank, 2026-10-02, with step 6.3's plan). Where emails come from and where notifications go sit above the key, so a business set up without them is never stuck until Settings (feature 12). Whenever the card is saved and a key is there (the new one, or the one already saved), one test email goes from the new sender to the new notification address first; refused, nothing is

saved. Phone, website and colour stay for Settings: nothing is stuck without them. 10. **A booking the owner makes emails the customer, never the business** (Frank, 2026-10-02, open question 4). The customer gets the same confirmation and invite as from the booking form, when an email was given. The business gets no notification: the owner made it, and it is already on their calendar and in their leads. The owner's screen (feature 11) can add a "don't send" box later. 11. **The booked worker gets no email in this feature** (Frank, 2026-10-02, open question 5). A worker with Google already sees the event in their calendar (feature 5); one without is told by text with feature 8, the fallback decision 8 put in the build plan.

Out of scope

- Texts, reminders and retrying a failed email: feature 8 (the job runner).
- The cancel and reschedule link in the confirmation: feature 7.
- Telling the booked worker, beyond what their Google calendar shows (decision 11): a text with feature 8.
- Editing the business details from the dashboard: feature 12 (Settings).
- Inbound email, CRM email and the BCC capture address: Phase 6.
- Each business's own booking questions: features 9 and 12.

Build loop

Steps are built one at a time on `feature/06-confirmations`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

Installing a package is a line only Frank crosses: steps 6.1 and 6.5 name theirs in their plan, and his yes to that plan is the yes to install them.

Build steps

- ☒ **6.1 The email seam, and login codes through it.**
 - Install `resend` in `backend` (Frank's yes with this plan).
 - `backend/lib/email/send-email.ts: sendEmail(...)` (Data / contracts). It sends through Resend with the key it is given (the business's, or the agency's `RESEND_API_KEY` for login codes), with a 10-second timeout, and returns Resend's id; Resend reports failures in its answer rather than by throwing, so an error answer throws one safe reason (never the key, the addresses, the

subject or the body). Without a key, outside production, it prints one line with the kind and the idempotency key only, and returns `null`.

- `backend/lib/email/read-email-settings.ts` reads and checks the settings; `server.ts` calls it at start, so production without `RESEND_API_KEY` or the login sender (`LOGIN_EMAIL_FROM`, decision 7) refuses to start, like `readTokenKey()`.
- `send-login-code.ts` sends the code through `sendEmail` from the login sender, plain and short; it keeps printing to the console in development without a key, and never puts the code in a log line otherwise.
- `.env.example`: `RESEND_API_KEY` now read; the login sender named.
- **Done when** saved tests, with Resend's `fetch` faked: a send carries the from, to, reply-to, subject, both bodies, the attachment and the idempotency key, with the key it was given; Resend's error answer throws a reason with no key, address or content in it; a timeout throws; no key in development prints one line without content and sends nothing; the settings check refuses production without the key or the login sender; a login code is sent through the seam from the login sender. Both builds pass. Resend's documentation checked: a "Sending access" key cannot read emails (the answer goes in the log).
- ☒ **6.2 The business details an email needs, and its locked key.**
 - Migration 0016 adds to `organization` (decision 8): `senderEmail` (the confirmation's from address), `notifyEmail` (where the business notification goes; client setup fills it with the owner's email to start, decision 8), `phone`, `website`, `brandColor` (a `#rrggbb` hex). All nullable at the database; the sending refuses without the two addresses and logs it (decision 4).
 - The same migration adds `email_sending_key`: `organizationId` (primary key, cascade), `credentials` (the key, locked with `encryptCredentials` bound to the business), `savedAt`. Read only by the sending code; no route returns it (decision 6).
 - `backend/lib/email/save-email-sending-key.ts`: checks the key's shape (`re_` then letters, digits and underscores), sends a test email with it from the business's sender to its notification address, and only then stores it locked, replacing any earlier key. A refused test answers why in safe words ("Resend refused this key", "The sender's domain is not verified in Resend") and stores nothing.
 - A shared Zod schema for them (emails lowercased and trimmed, the phone 1 to 40 characters, the website an `https://` address, the colour `#rrggbb`), used by client setup.
 - Client setup (`POST /admin/clients`, its schema and the 3b form) takes them, and the key optionally (path A, decision 6), saved through the same function; the seed fills both dev businesses with addresses at `example.com` and no key.
 - `backend/lib/email/find-business-email-details.ts` reads them, with the name, logo and time zone, inside one business only.
 - **Done when** saved tests, with Resend faked: setup stores the details and refuses a bad colour, a website without `https`, an address that is not an email; a key is stored locked only after its test email went, and a refused test stores nothing; the stored key never appears in any route's answer

or log line; the reader returns one business's details and never another's; both builds pass; the 3b form shows the new fields with their labels and errors (checked by hand in the browser, the screenshot in the log).

- **6.3 The owner's "Email sending" card.**

- `PUT /email-sending` (dashboard route: `requireOrganizationMiddleware`, `requirePermissionMiddleware({ organization: ["update"] })`, the dashboard CORS, CSRF and no-store middleware): takes `{ senderEmail, notifyEmail, key? }` (decision 9); with a key, new or already saved, the test email goes first and a refusal saves nothing; answers the card's state or a refusal. The business is the signed-in one, never from the request.
- `GET /email-sending`: the two addresses and when a key was saved; never the key.
- `frontend/components/email-sending/email-sending-card.tsx` on the dashboard home, beside the calendar card: "Not set up" or "Sending from bookings@primopainters.com, key saved Oct 2"; the two address fields; a password-type field to paste a new key, a Save button that locks while saving, the refusal shown under the field and read out, the field emptied after a save. Called through the typed dashboard client.
- **Done when** saved tests: an owner saves the two addresses and a key, and the answer never carries the key; changing an address with a key already saved sends the test from the new address, and a refusal keeps the old addresses; a member without the permission is refused; another business's key is never touched; `GET` never carries the key; a refused test answers its safe reason and stores nothing. The frontend build passes and the typed client sees both routes; the card checked by hand in the browser (not set up, a refused key, a saved key), the screenshots in the log.

- **6.4 The calendar invite.**

- `backend/lib/email/booking-ics.ts`: one pure function from a booking (id, service name, start, end, address, business name, sender email, customer name and email) to the text of an `.ics` file: `METHOD:REQUEST`, one `VEVENT` with `UID:<booking id>` (a random UUID; never the sender's domain, which the owner can change), `SEQUENCE:0`, `DTSTAMP`, `DTSTART` and `DTEND` in UTC (`...Z`), `SUMMARY "<service> with <business>"`, `LOCATION` the address, `ORGANIZER` the business, `ATTENDEE` the customer with `RSVP=FALSE`, `STATUS:CONFIRMED`. CRLF line ends, lines folded at 75 octets without splitting a character, and commas, semicolons, backslashes and new lines escaped (decision 5).
- **Done when** saved tests: the exact text for a plain booking; an address with a comma, a semicolon and a new line comes out escaped; a long Spanish service name folds at 75 octets and unfolds back to the same text; the UID is the same for one booking every time (feature 7 cancels and moves the same event by it).

- **6.5 The two emails.**

- Install `@react-email/components`, `react` and `react-dom` in `backend`, and `@types/react` as a dev dependency (Frank's yes with this plan; Primo's pattern). `render` comes inside `@react-email/components`, so `@react-email/render` is not installed on its own. The templates are

`.tsx` files with `/* @jsxImportSource react */`, because the backend's own JSX setting is Hono's.

- `backend/emails/booking-confirmation.tsx`, to the customer: the business's logo (alt text its name) and colour; "You're booked"; the service, the day and time in the business's zone with the zone named, the person, the address; a `tel:` button for the business's phone; the business's website; a plain-text twin. Never the product's name.
- `backend/emails/booking-notification.tsx`, to the business: "New booking: <service>, <day and time>", then the customer's name, email, phone, address and words, each scannable at a glance, and the person booked. Reply-to is the customer's email when given, so pressing Reply writes to them.
- Everything the customer typed is rendered as text by React, never as HTML. Dates go through one formatter (`format-booking-time.ts`) that takes the business's zone.
- `npm run email:preview --workspace=backend` writes both, filled with a sample booking, to a gitignored `backend/.email-preview/` folder to open in a browser; nothing is sent.
- **Done when** saved tests: each email renders with the right subject, the time in the business's zone (a booking at 15:00Z reads 9:00 a.m. in Edmonton), the `tel:` link, no product name, and a customer's `<script>` shown as text; the plain-text twins carry the same facts; the preview writes two files; both opened by hand, the screenshots in the log.
- ☒ **6.6 Sent once the booking is saved.**
 - `backend/lib/booking/booking-confirmation-emails.ts`, beside `booking-event-writes.ts` and in the same shape: started by `bookTime` after the booking's transaction, not awaited; the sends still running can be awaited by tests. It reads the booking and the business details inside one business, renders the two emails, attaches the invite to the customer's, and sends with the business's own key, to the recipients one function decides (`findBookingEmailRecipients` : the business's notification address today, decision 8): the customer's only when an email was given, the business's always. Idempotency keys `booking-confirmation/<booking id>` and `booking-notification/<booking id>`.
 - Each email that went gets an `email_sent` timeline entry on the contact (Data / contracts), never an address or content.
 - Only a new booking sends: a resent form (`alreadyBooked`) sends nothing. A failure keeps the booking and logs one line with the booking id and a safe reason; a business without its two addresses or its key logs that and sends nothing (decision 4). Retrying is feature 8.
 - A booking the owner makes (source `manual`) sends only the customer's confirmation (decision 10). The booked worker gets no email (decision 11).
 - **Done when** saved tests, with Resend faked: a booking with an email sends both, the invite attached to the customer's, and writes two timeline entries; a phone-only booking sends only the business's; a resent form sends nothing more; Resend failing keeps the booking, logs one line with no address, and writes no entry; a business missing its addresses or its key sends nothing and logs it; the route's answer does not wait for the emails (the same held-answer test as 5d's Google event); no log line or timeline payload carries a customer's name, email, phone, address or words.

Files / areas

- `backend/package.json` (resend; React Email, react, react-dom)
- `backend/lib/email/` (new): `send-email.ts`, `read-email-settings.ts`, `check-email-sending-key.ts`, `store-email-sending-key.ts`, `save-email-sending.ts`, `find-email-sending-state.ts`, `email-refusal-code.ts`, `booking-ics.ts`, `find-business-email-details.ts`, `format-booking-time.ts`
- `backend/routes/email-sending-routes.ts` (new), `backend/app.ts`
- `frontend/components/email-sending/` (new), the dashboard home, `frontend/lib/api-client.ts`
- `backend/emails/` (new): the two templates and the preview script
- `backend/lib/auth/send-login-code.ts`, `backend/server.ts`
- `backend/lib/booking/book-time.ts`, `booking_confirmation_emails.ts` (new)
- `backend/lib/admin/provision-client.ts`, `backend/routes/admin-routes.ts`
- `packages/shared/db/auth-tables/organization-table.ts`, a new `email-sending-key` table, migration 0016, `zod-validation/` (the business details, client setup), `scripts/seed-dev.ts`
- `frontend/app/admin/clients/new/` (the 3b form's new fields)
- `.env.example`, `.gitignore`
- their tests

Data / contracts

organization gains (6.2): `senderEmail` text null, `notifyEmail` text null, `phone` text null, `website` text null, `brandColor` text null (`#rrggbb`). `logo` exists (an absolute `https://` image URL when set).

email_sending_key (6.2): `organizationId` text primary key (cascade), `credentials` text not null (`encryptCredentials(key, readTokenKey(), organizationId)`), `savedAt` timestamptz. One key per business.

The card's routes (6.3, decision 9): `PUT /email-sending` body `{ senderEmail: string; notifyEmail: string; key?: string }` answers 200 `{ senderEmail, notifyEmail, keySavedAt }`, 400 `bad_request` (an address that is not an email, not a Resend key), 422 `key_refused` with the safe reason, 401/403 as the other dashboard routes; `GET /email-sending` answers the same three (each may be null).

sendEmail (6.1):

```
type SendEmailInputType = {
  apiKey: string | null; // the business's or the agency's key; null only in developme
  kind: string; // for the log line only: "login_code", "booking_confirmation", ...
  from: string; // "Primo Painters <bookings@primopainters.com>"
```

```

to: string[];
replyTo?: string;
subject: string;
html: string;
text: string;
attachments?: { filename: string; content: string; contentType: string }[];
idempotencyKey: string; // the same key never sends twice (Resend keeps it 24 hours)
};
// Resolves to Resend's email id, or null when nothing was sent (development without a
// Throws Error("Sending an email failed: <safe reason>") otherwise.

```

The invite (6.4): file name `invite.ics`, content type `text/calendar; charset=utf-8; method=REQUEST`.

The timeline entry (6.6): `type: "email_sent"`, payload `{ bookingId, kind: "booking_confirmation" | "booking_notification", resendId }`, `actorUserId` null.

Testing

Backend Vitest. The invite builder and the date formatter are pure. The seam, the emails and the sending use Resend's `fetch` faked: no test ever sends a real email. Database tests build businesses of their own and remove them, and wait for running sends before closing the database. Each case on the feature's Simulate page ("Who hears what when Jane books?") is a saved test under the same name, in the step that builds its rule (6.6). No real email is sent until Frank sets a Resend key and a verified domain on purpose.

Notes for the AI

- Resend's SDK returns `{ data, error }` and does not throw on an API error; check `error`.
- Resend's idempotency option exists on `emails.send`; confirm the installed version's exact option name before relying on it.
- The backend's `tsconfig` sets `jsxImportSource: "hono/jsx"`; the email templates override it per file. Check that `tsc` builds them into `dist/`.
- Logo URLs must be absolute; most clients block images, so the alt text is the business's name.
- Dates: `Intl.DateTimeFormat` with the business's `timeZone`, never the server's zone (Railway runs UTC).
- Never put a customer's details or a login code in a log line.
- For 6.6, from step 6.5's review: `findBusinessEmailDetails` gives `timezone: string | null` while the templates need a string, and `Intl` throws on a missing zone, so decide what a business without one does. The notification's footer says replying reaches the customer, which holds only once

`replyTo` is set to their email. Send the invite so Outlook on a desktop sees a meeting (step 6.4's review). Alberta keeps daylight time for good in the time zone data from 2026c (Node 26 here); check that the Node on Railway carries the same data, or Primo's winter times read an hour off.

Open questions

Each blocks only the step named; Frank answers it when that step's plan is gone through, before it is built.

1. Answered 2026-10-02: decision 6. 2. Answered 2026-10-02: decision 8. 3. Answered 2026-10-02: decision 7. 4. Answered 2026-10-02: decision 10. 5. Answered 2026-10-02: decision 11.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/auth-tables/organization-table.ts` : the business gains `senderEmail` , `notifyEmail` , `phone` , `website` and `brandColor` ; `logo` already existed. A business without both addresses sends nothing (decision 4).
- `db/auth-tables/email-sending-key-table.ts` : `email_sending_key` , one row per business, the key locked with the token cipher and bound to the business id. Its own table, not a column on `organization` , because Better Auth's own routes return the organization to its members (decision 6).
- Migration `0016_business_email_details.sql` , generated and untouched.
- `zod-validation/organization-validation-schemas/` : the business's email details (an https website, a `#rrggb` colour, an email-shaped sender), a Resend key's shape (`re_` then letters, digits and underscores), and the owner's card (`senderEmail` , `notifyEmail` , optional `key`). Client setup (3b) takes the same details; an empty notification address falls back to the owner's email (decision 8).
- `scripts/seed-dev.ts` : both dev businesses get example addresses and no key, so development never sends a real email.

backend: the email seam and login codes

- `lib/email/send-email.ts` : the one door every email goes out through, with the key it is given. Every send carries an idempotency key, so a repeat never makes a second email. In development without a key it prints one line with ids only; in production a missing key throws. Failures throw `SendEmailError` carrying only Resend's error name and status, never its message, which can repeat the addresses.

- `lib/email/read-email-settings.ts` : the agency's own `RESEND_API_KEY` and `LOGIN_EMAIL_FROM` (decision 7). `server.ts` calls it at start, so production refuses to start without them, the way it already refuses without the calendar token key.
- `lib/auth/send-login-code.ts` and `auth-server.ts` : login codes go through the seam from the agency's address. The sign-in route does not wait for the email, so a known address answers as fast as an unknown one and the form cannot be used to find out who is a client (F-97).

backend: each business's own key

- `lib/email/check-email-sending-key.ts` : a key is proved by sending one test email with it, from the business's sender to its notification address. A refusal comes back as `key_refused`, `sender_refused` or `email_test_failed` (`email-refusal-code.ts`, the codes in `lib/errors/refuse.ts`), so each form shows it under the right field.
- `lib/email/store-email-sending-key.ts` : keeps the key locked, replacing any earlier one, only after the test passed.
- `lib/email/save-email-sending.ts` and `find-email-sending-state.ts` : the owner's card saves the two addresses and, when pasted, a new key (decision 9). With a key there, the new one or the one already saved, one test email goes from the new sender first; refused, nothing changes. The state read never selects the key, not even locked.
- `lib/email/find-business-email-details.ts` : the only place a key is unlocked, inside its own business, for the sending code.
- `routes/email-sending-routes.ts` : `GET` and `PUT /email-sending`, the business from the session only, saving only for a role that may change the business. The dashboard CORS now allows `PUT` for it.
- `lib/admin/provision-client.ts` and `routes/admin-routes.ts` : client setup stores the business's email details and, when given, tests and stores its key.

backend: the invite and the two emails

- `lib/email/booking-ics.ts` : one booking as an RFC 5545 invite. Times in UTC (decision 5); the UID is the booking's own id alone, never the sender's domain, which the owner can change (F-107); text escaped and control characters dropped (F-109); lines folded at 75 octets without splitting a character. DTSTAMP is passed in, and the sender stamps it with the booking's creation time, so a retry sends the very same invite under the same key.
- `lib/email/format-booking-time.ts` : every time in the business's own zone, the zone named, written the Canadian way: "Thursday, October 8 at 9:00 a.m. MDT".
- `emails/` : the two React Email templates, the customer's confirmation and the business's notification, and their shared parts (the frame, a labelled line, the button, hex colours, the `tel:` link, the facts both take). Each file says `/* @jsxImportSource react */`, because the backend's own JSX is Hono's. `render` comes from `@react-email/components`; the plain-text twin is `render(..., { plainText: true })`. What the customer typed is always text. The product's name appears in neither.

- `scripts/email-preview.ts` : `npm run email:preview --workspace=backend` writes both emails, filled with a sample booking, to the gitignored `backend/.email-preview/`. React Email's own preview server was left out on purpose: one more package for a few lines.

backend: sent once the booking is saved

- `lib/email/find-booking-email-recipients.ts` : who hears, in one place. The customer when they gave an email; the business's notification address unless the owner made the booking (`lead.source` `manual`, decision 10); never the booked worker (decision 11). A person's own work email joins here with feature 12.
- `lib/email/send-booking-emails.ts` : reads the booking and the business inside that business, sends nothing for a cancelled booking or a business missing its addresses, key or time zone (one plain log line), renders both emails, attaches the invite to the customer's, and sends each with the business's own key under `booking-confirmation/<id>` or `booking-notification/<id>`. One email failing never stops the other; failures make one warning line per booking, ids and reasons only. Each email that went gets an `email_sent` timeline entry with the booking, the kind and Resend's id.
- `lib/booking/booking-confirmation-emails.ts` : started by `bookTime` after the booking's transaction, beside the Google event and in its shape: not awaited, its failures caught (F-113), the running sends awaitable for tests. Only a new booking starts it; a resent form returns before it.

frontend

- `components/email-sending/email-sending-card.tsx` : the dashboard home's "Email sending" card. "Not set up", or the sender and the date a key was saved; the two address fields; a password field to paste a new key, emptied after a save; each refusal under its own field and read out.
- `components/admin/new-client-form.tsx` : client setup's "Their emails" section.
- Both through the typed dashboard client in `lib/api-client.ts`.

Packages

Installed on Frank's yes to each step's plan: `resend` (6.1), and `@react-email/components`, `react`, `react-dom` and the dev dependency `@types/react` (6.5). `@react-email/render` was not installed: it comes inside `@react-email/components`. One copy of React, shared with the frontend.

Tests

Every case on the feature's Simulate page is a saved test under the same name. Resend is faked in every test that could reach it; no test sends a real email, and no business in the dev database has a key. The booking tests that book now wait for both the Google event and the emails before closing their database. Final count: 450 backend and 103 shared tests.

Review history

Every step was reviewed by a fresh reviewer before the next began; no P0 or P1 was ever open. The findings of each review (F-96 to F-115) were fixed before the next step, and the final review below passed over the whole feature, closed the seventeen still marked fixed, and raised one P3, F-116, carried forward.

Carried forward

- F-116 (P3): finding numbers in three code comments from step 6.1.
- No real email has been sent: the first waits for Frank to set up a business's Resend key and verified domain on purpose. Outlook's handling of the invite as a meeting is only checked then.
- At deploy: Railway with `NODE_ENV=production` (the start-up refusal, the seam's no-key refusal and the Resend SDK's quiet errors all depend on it); `RESEND_API_KEY` and `LOGIN_EMAIL_FROM` set; migration 0016; Node with time zone data 2026c or newer, where Alberta keeps daylight time for good.
- Feature 5's Google event send has the same untested failure guard F-113 closed for emails.
- Feature 8: a restart between a booking's save and its sends loses those emails until its job runner retries them. Feature 9: rate limits matter more now that a public booking sends email to any address typed in.
- For a clinic, "Where" shows the address the customer typed, from feature 5's booking; a clinic visit may happen elsewhere.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-58, F-62, F-94, F-95, F-116 stay in the live ledger.

6/F-96 [P2] closed - With the agency's Resend settings in the root .env, the route tests send real login emails and then fail

File: backend/routes/admin-routes.test.ts:38 and backend/routes/calendar-routes.test.ts:54 (path: backend/lib/auth/send-login-code.ts:21-32) **Found:** 2026-10-02 by independent review of step 6.1 (scope: 7dc0721..de30223; lenses: quality, security, performance, tests) **Why it matters:** Both route test files load the root `.env` and sign in by reading the code `sendLoginCode` prints. Since 6.1, once `RESEND_API_KEY` and `LOGIN_EMAIL_FROM` are set there (the same file the app reads, and the way to try real login emails locally), `sendLoginCode` sends through Resend instead of printing: every signed-in test user gets a real email from the agency's account to an `@example.com` address (bounces count against the agency's sending domain), no code is printed, and the whole file fails. Reproduced in this review without leaving the machine, by setting both values plus `RESEND_BASE_URL=http://127.0.0.1:9`: Better Auth logged "Sending an email failed", the helper threw "No login code was printed for admin-frank-...@example.com", and all 25 admin route tests were skipped. The spec's Testing section says no

test ever sends a real email. **Suggested fix:** In both `signIn` helpers (or a shared one), clear the two settings for the request with `vi.stubEnv("RESEND_API_KEY", "")` and `vi.stubEnv("LOGIN_EMAIL_FROM", "")` (read per send, so this works), or fake `fetch` for `api.resend.com` there. **Resolution:** Fixed in 6.1's review fixes: `admin-routes.test.ts` and `calendar-routes.test.ts` drop `RESEND_API_KEY` and `LOGIN_EMAIL_FROM` right after loading `.env`, so their sign-in helper always reads the code from the console. Shown with `RESEND_API_KEY`, `LOGIN_EMAIL_FROM` and `RESEND_BASE_URL` (a dead local port) set: all 65 route tests pass and nothing is sent. Closed 2026-10-02 by independent review of step 6.2: `admin-routes.test.ts` (in this step's scope) and `calendar-routes.test.ts` still drop both settings before the app loads, and 6.2's new admin tests fake `fetch` for the key's test email, so nothing leaves the machine; all 393 backend tests pass.

6/F-97 [P2] closed - A known address now waits for Resend while an unknown one answers at once, so the sign-in form tells who is a client

File: `backend/lib/auth/auth-server.ts:155-157` (comment at :148-150; send: `backend/lib/auth/send-login-code.ts:32`) **Found:** 2026-10-02 by independent review of step 6.1 (scope: 7dc0721..de30223; lenses: quality, security, performance, tests) **Why it matters:** `auth-server.ts` says an unknown address is told a code is on its way "so the form can't be used to test who is a customer". Better Auth 1.7.5 returns at once for an unknown address, but for a known one it awaits `sendVerificationOTP` (no `backgroundTasks` handler is set, so `runInBackgroundOrAwait` awaits it), and since 6.1 that is a round trip to `api.resend.com`, hundreds of milliseconds and up to the 10-second limit. Before 6.1 the known path only printed, so the answer took the same time either way. One timed request per address now shows whether that person is one of the agency's clients; the 3-per-minute rate limit slows a sweep but not a targeted check. **Suggested fix:** Answer before the email goes: in `sendVerificationOTP`, start `sendLoginCode` without awaiting it and catch its failure into one safe log line, or set Better Auth's `advanced.backgroundTasks.handler`. Either way the owner's experience is unchanged, since a failed send is already swallowed and answered as success. **Resolution:** Fixed in 6.1's review fixes: `auth-server.ts` starts `sendLoginCode` without awaiting it, logging a failure as a safe reason, so a known address answers as fast as an unknown one. Test: `login-code-timing.test.ts` holds Resend and gets the 200 first; with the `await` put back it times out. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): `auth-server.ts` (in scope) starts `sendLoginCode` without awaiting it and logs only a safe reason, so a known address answers as fast as an unknown one. Putting the `await` back fails `login-code-timing.test.ts`.

6/F-98 [P3] closed - Nothing fails if the login code is written to a log line on the sending path

File: `backend/lib/auth/send-login-code.test.ts:28` (code: `backend/lib/auth/send-login-code.ts:29-41`) **Found:** 2026-10-02 by independent review of step 6.1 (scope: 7dc0721..de30223; lenses: quality, security, performance, tests) **Why it matters:** The step's plan says the code is never put in a log line outside development without settings, and the spec's Notes repeat it. In this review a `console.log` of the code was added just before `sendEmail` and all 17 email and login-code tests still passed. A

debugging line left in would put live sign-in codes into Railway's logs with nothing to catch it. **Suggested fix:** In the "goes through the door" test, spy on `console.log` and `console.error` and assert no call contains the code. **Resolution:** Fixed in 6.1's review fixes: the sending-path test spies console.log, info, warn and error and fails if the code appears in any line. Proved: a debug console.log of the code makes it fail. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): a console.log of the code added just before sendEmail fails send-login-code.test.ts.

6/F-99 [P3] closed - The sendEmail contract says apiKey is always a string, and the overview still says login codes cannot be sent in production

File: blueprint/context/current-feature.md:307; blueprint/context/project-overview.md:220 **Found:** 2026-10-02 by independent review of step 6.1 (scope: 7dc0721..de30223; lenses: quality, security, performance, tests) **Why it matters:** The code takes `apiKey: string | null` (null meaning development without a key, as the 6.1 bullet itself describes), but the Data / contracts block that 6.6 will build against still types it `string`. The overview's risk line "Login codes cannot be sent in production until email exists (item 6)" is no longer true after this step. Both send the next reader to the wrong shape. **Suggested fix:** Type the contract's `apiKey` as `string | null` with the same note as the code; drop or rewrite the overview line when the overview is next refreshed. **Resolution:** Fixed in 6.1's review fixes: the spec's sendEmail contract types apiKey as string | null; the overview says login codes go by email from the agency's address and names LOGIN_EMAIL_FROM. Closed 2026-10-02 by independent review of step 6.2: the contract in current-feature.md (in this step's scope) types apiKey as string | null and still matches send-email.ts, whose SendEmailError keeps the "Sending an email failed: <reason>" message; the overview line names LOGIN_EMAIL_FROM.

6/F-100 [P3] closed - Resend refusals that will never pass on a retry are told as "Try again shortly"

File: backend/lib/email/check-email-sending-key.ts:31 (the from line: :41) **Found:** 2026-10-02 by independent review of step 6.2 (scope: 7e287c0..8e181e9; lenses: quality, security, performance, tests) **Why it matters:** reasonFor names 401, a code ending in api_key and 403; every other answer says "Resend could not send the test email just now. Try again shortly." The installed SDK's own list of codes includes invalid_from_address and invalid_parameter (422), daily_quota_exceeded and monthly_quota_exceeded, none of which a retry fixes, so Frank (and in 6.3 the owner) retries a setup that can never pass, with no hint why. One way to get there is unverified: the from line puts the business name in unquoted (`Smith, Jones & Co <bookings@...>`), and a comma or angle bracket in a display name is not a valid address header unless quoted; the business name schema allows both. **Suggested fix:** Keep "try again shortly" for the timeout, the 429 rate limit and 5xx only; answer invalid_from_address and the other 422s as a problem with the sender address, and the quotas as the Resend account's sending limit. Quote the display name (escaping " and \) in the from line, which 6.6 will build the same way. **Resolution:** Fixed in 6.2's review fixes: a sending limit and a sender address Resend will not take (422, invalid_from_address) now get their own plain reasons; only what a retry can fix says to try again. The

business name is quoted in the from line (format-sender.ts), so a name with a comma or an ampersand stays one name. Tests: the two new refusals, and a name with a comma, an ampersand and quotes. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): check-email-sending-key.ts (in scope) answers a sending limit and a sender address Resend will not take in their own words, and format-sender.ts quotes the name. Removing the quota branch fails a test. One weak spot, not a defect: deleting the `status === 422` half of the sender condition leaves all 8 tests green, because the only 422 case is `invalid_from_address`, which the name check also catches.

6/F-101 [P3] closed - An unreadable saved key blocks saving the new key that would replace it

File: backend/lib/email/save-email-sending-key.ts:16 (decrypt: backend/lib/email/find-business-email-details.ts:51) **Found:** 2026-10-02 by independent review of step 6.2 (scope: 7e287c0..8e181e9; lenses: quality, security, performance, tests) **Why it matters:** saveEmailSendingKey starts by calling findBusinessEmailDetails, which unlocks the stored key. When that key cannot be unlocked (CALENDAR_TOKEN_KEY changed, or a row restored onto another business, which email-sending-key.test.ts shows throws), the save throws before the test email, so pasting a new key in the 6.3 card, the only way to repair it, fails with a 500 every time. The calendar code treats keys it cannot open as replaceable (hand-back-calendar-permission.ts returns "not_confirmed" instead of throwing). **Suggested fix:** Read only the name and the two addresses in saveEmailSendingKey (a select of its own, or a reader option that skips the key); the old key is overwritten, never needed. **Resolution:** Fixed in 6.2's review fixes: saveEmailSendingKey reads only the name and the two addresses, never the stored key, so a key that can no longer be unlocked is still replaced. Test: a broken stored key is replaced by a new one. Closed 2026-10-02 by independent review of step 6.3: save-email-sending-key.ts is gone, and its successor backend/lib/email/save-email-sending.ts (in this step's scope) opens the saved key only when no new key is pasted (keyToTest returns a new key first), so a new key still replaces one that cannot be unlocked; with no new key, an unreadable one is answered "Paste it again" instead of a 500. Both cases are tests in email-sending-key.test.ts and pass.

6/F-102 [P3] closed - The test of a wrong key fakes 401, so the branch that catches Resend's real wrong-key answer is untested

File: backend/lib/email/check-email-sending-key.test.ts:49 (code: backend/lib/email/check-email-sending-key.ts:24) **Found:** 2026-10-02 by independent review of step 6.2 (scope: 7e287c0..8e181e9; lenses: quality, security, performance, tests) **Why it matters:** Resend answers an invalid key with `invalid_api_key` and status 403, the same status as an unverified domain, so it is the `error.code.endsWith("api_key")` half of the condition that keeps a wrong key from being told to verify its domain. In this review that half was deleted and all 42 tests in check-email-sending-key, email-sending-key and admin-routes still passed (reverted after). **Suggested fix:** Add a case with name `invalid_api_key` and status 403 expecting "Resend refused this key.", and keep the 401 case for a missing or restricted key. **Resolution:** Fixed in 6.2's review fixes: the error name is checked first, and the wrong-

key test now fakes Resend's real answer (invalid_api_key with 403), beside a missing key (401). Proved: without the name check the 403 test fails. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): removing the `api_key` name check fails the invalid_api_key 403 case.

6/F-103 [P3] closed - A phone of only spaces shows Zod's raw English under the field

File: packages/shared/zod-validation/organization-validation-schemas/business-email-details-validation-schema.ts:15 **Found:** 2026-10-02 by independent review of step 6.2 (scope: 7e287c0..8e181e9; lenses: quality, security, performance, tests) **Why it matters:** The phone is trimmed, then `.min(1)` has no message, so " " is refused with "Too small: expected string to have >=1 characters" (seen in this review through the built schema), and the 3b form shows that under Phone, unlike every other field's plain message. **Suggested fix:** Give `.min(1, ...)` a plain message, or treat a phone of only spaces as empty, the way the other optional fields read. **Resolution:** Fixed in 6.2's review fixes: a phone of only spaces answers "Enter a phone number, or leave it empty." Test added. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): taking the plain message off the phone's `.min(1)` fails the shared schema test.

6/F-104 [P3] closed - A refusal about the sender address is shown under the key field, even when no key was pasted

File: frontend/lib/api-client.ts:236 (refusal: backend/routes/email-sending-routes.ts:53; focus: frontend/components/email-sending/email-sending-card.tsx:119) **Found:** 2026-10-02 by independent review of step 6.3 (scope: 41d8fd8..5e79646; lenses: quality, security, performance, tests) **Why it matters:** Every reason from the test email comes back as one code, `key_refused`, and `saveEmailSending` puts every `key_refused` under the "Resend key" field and focuses it. Two of those reasons are about the address, not the key: "Resend would not send from this address..." (a 403 for an unverified domain) and "Resend would not accept this sender address...". Decision 9's own case shows it: an owner with a key saved changes "Emails come from" to a new domain and leaves the key empty; the refusal (the one `email-sending-key.test.ts` asserts for exactly this case) lands in red under the empty key field, so the owner is pointed at the key and may paste a new one, when the fix is the sender's domain. **Suggested fix:** Let the backend say which field a reason is about (a second code such as `sender_refused`, or a `field` on the refusal), and put sender refusals under "Emails come from" and account-wide ones (the sending limit, "try again shortly") in the card's notice. **Resolution:** Fixed in 6.3's review fixes: `checkEmailSendingKey` now says what a refusal is about (the key, the sender, or neither), and both forms get a code for each: `key_refused` under the key field, `sender_refused` under "Emails come from", `email_test_failed` as a notice (`email-refusal-code.ts`). Tests: each refusal's subject in the check's table; a sender refusal on the card answers `sender_refused`. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): mapping sender refusals to `key_refused` in `email-refusal-code.ts` fails `email-sending-routes.test.ts`; the card and the 3b form each put `sender_refused` under the sender field.

6/F-105 [P3] closed - Nothing fails if the Email sending routes lose their plan checks

File: backend/routes/email-sending-routes.test.ts:154 (checks: backend/routes/email-sending-routes.ts:23-24, :35-36) **Found:** 2026-10-02 by independent review of step 6.3 (scope: 41d8fd8..5e79646; lenses: quality, security, performance, tests) **Why it matters:** Both routes mount `requireKnownSubscriptionMiddleware` and `requireModuleMiddleware("booking")`, like the calendar card's routes. In this review both were removed from GET and PUT and all 18 tests in `email-sending-routes.test.ts` and `email-sending-key.test.ts` still passed (reverted after). `calendar-routes.test.ts` proves the same checks with a business on a plan without booking (a 403); here a later edit could let a business without the booking module set up sending, with nothing to catch it. **Suggested fix:** Add the calendar tests' no-booking tenant (an unrecognised plan stands in until a real tier lacks booking) and expect 403 from GET and PUT, with no test email asked for. **Resolution:** Fixed in 6.3's review fixes: a business on a plan without booking is refused reading and saving the card, with nothing sent. Proved: with both plan checks removed the test fails. The booking check alone cannot be told apart from the known-plan check until a real tier lacks booking (feature 23), as on the calendar card. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): removing both plan checks from GET and PUT fails the no-booking-plan test.

6/F-106 [P3] closed - A step number in a code comment, and the spec still names the removed `save-email-sending-key.ts`

File: backend/middleware/dashboard-middleware/dashboard-cors-middleware.ts:12; blueprint/context/current-feature.md:297 **Found:** 2026-10-02 by independent review of step 6.3 (scope: 41d8fd8..5e79646; lenses: quality, security, performance, tests) **Why it matters:** `coding-standards.md` says no history in code comments, step numbers named first; the new CORS comment ends "(6.3)". The spec's Files / areas list, which /complete and later steps read, still names `save-email-sending-key.ts`, removed in this step, and not `save-email-sending.ts` or `find-email-sending-state.ts`, which replaced it. **Suggested fix:** Drop "(6.3)" from the comment (the reason, saving the Email sending card, can stay). In Files / areas, name the two new files in place of the removed one. **Resolution:** Fixed in 6.3's review fixes: the step number is out of the CORS comment, and the spec's Files / areas names the files that replaced `save-email-sending-key.ts`. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): the CORS comment carries no step number, and the spec's Files / areas names `save-email-sending.ts` and `find-email-sending-state.ts`.

6/F-107 [P2] closed - The invite's UID changes when the business changes its sender address

File: backend/lib/email/booking-ics.ts:18-20, :28 (the plan: blueprint/context/current-feature.md, step 6.4) **Found:** 2026-10-02 by independent review of step 6.4 (scope: adb473e..6febd01; lenses: quality, security, performance, tests) **Why it matters:** The UID is the booking id plus the domain of `senderEmail`, and that address is editable: the owner's card saves a new one (`backend/lib/email/save-email-sending.ts:78`, decision 9), and decision 6's handover changes it by design (Frank builds with a test sender

of his own, the owner then sets their own). A booking confirmed before the change and moved or cancelled by feature 7 after it would carry a different UID, so the customer's calendar would get a second event instead of moving the first. The step's own promise ("the UID is the same for one booking every time") is only tested against a different DTSTAMP (booking-ics.test.ts:93), never a different sender.

Suggested fix: Build the UID from the booking id alone (already a `randomUUID()` in book-time.ts:281, so globally unique; RFC 5545 does not require an `@domain`), or from a part that never changes and never names the product. Amend step 6.4's UID line in the spec to match, and add a test that a different `senderEmail` gives the same UID. **Resolution:** Fixed 2026-10-02: the UID is now the booking's own id alone (a random UUID), never the sender's domain; the spec's step 6.4 line amended to match. The UID test also expects the same UID from a different sender, and the exact-text test reads `UID:bk_123`. Putting the domain back fails two tests. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): putting the sender's domain back in the UID fails two tests in booking-ics.test.ts.

6/F-108 [P3] closed - The fold tests never fill a continuation line or carry an emoji

File: backend/lib/email/booking-ics.test.ts:73-91 (code: backend/lib/email/booking-ics.ts:67-89) **Found:** 2026-10-02 by independent review of step 6.4 (scope: adb473e..6febd01; lenses: quality, security, performance, tests) **Why it matters:** Two mutations in this review left all 6 tests passing (both reverted). Raising the continuation limit from 74 to 75, which writes 76-octet lines, passed because the longest folded line in any test, the Spanish SUMMARY, is 143 octets: 75 on the first line and 69 on the second, so no continuation ever reaches the limit. Iterating UTF-16 code units instead of characters (`line.split("")`) also passed, though the comment at booking-ics.ts:69-70 promises an emoji arrives whole; only the two-octet "ó" is tested. The code itself is right today (probed: an emoji at the boundary moves whole, continuation lines are exactly 75 octets). **Suggested fix:** Make the Spanish (or another) value long enough for at least one full continuation line and expect a line of exactly 75 octets starting with a space; add a case with a four-octet emoji at the fold boundary that expects it whole on the next line. **Resolution:** Fixed 2026-10-02: two new tests, a continuation line filled to exactly 75 octets with its leading space, and a four-octet emoji at the fold point moving whole to the next line. Both mutations above (continuation limit 75, iterating UTF-16 code units) now fail a test. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): a continuation limit of 75 and iterating UTF-16 code units each fail a test.

6/F-109 [P3] closed - Control characters typed in the address reach the invite's LOCATION raw

File: backend/lib/email/booking-ics.ts:54-60 (input: packages/shared/zod-validation/booking-links-validation-schemas/create-booking-validation-schema.ts:22-26) **Found:** 2026-10-02 by independent review of step 6.4 (scope: adb473e..6febd01; lenses: quality, security, performance, tests) **Why it matters:** The booking form's address only trims and limits length, so a customer's typed address can hold control characters. Probed on the built function: an address of `12 Main<NUL>St<VT>Back<BEL>` comes out as `LOCATION:12 Main<NUL>St<VT>Back<BEL>`. RFC 5545's TEXT allows no control character but a

tab, so the attached invite is invalid and a strict parser may refuse all of it (a NUL can also cut the file short in C-based readers). No line can be injected: CR and LF are escaped. `quoteParameter` already strips controls from the two names; `escapeText` does not. A lone CR is handled but untested (removing `\r` from the regex left every test passing). **Suggested fix:** In `escapeText`, after the new-line escape, replace any remaining C0 control except tab, and DEL, with a space, as `quoteParameter` does; add a lone `\r` and a `\u0000` to the escape test. **Resolution:** Fixed 2026-10-02: `escapeText` replaces any control character left after the new-line escape, except a tab, with a space. The escape test now carries a lone `\r` and a `\u0000`; dropping the lone CR from the new-line escape fails it. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): removing the control-character replace from `escapeText` fails the escape test.

6/F-110 [P3] closed - The email tests run only in Edmonton, the laptop's own zone, so a template that ignores the business's zone passes

File: backend/emails/booking-emails.test.ts:9-28 (code: backend/emails/booking-confirmation.tsx:19, backend/emails/booking-notification.tsx:19) **Found:** 2026-10-02 by independent review of step 6.5 (scope: 67aaa74..fbaf9d3; lenses: quality, security, performance, tests) **Why it matters:** Every email test uses a business in `America/Edmonton`, and this laptop's own zone is `America/Edmonton`. Two mutations in this review left all 14 tests passing (both reverted): the confirmation calling `formatBookingTime` with a hard-coded `"America/Edmonton"`, and the notification calling it with the server's zone. The formatter's own test catches a formatter that drops its zone, but nothing ties the templates to `facts.business.timezone`, which is the step's Done when ("the time in the business's zone"); on Railway (UTC) a server-zone slip shows every booking six hours off. Two notification mutations also survived: the "Call <customer>" button pointed at `#` (the test titled "with a tel: link and button" is met by the Phone link alone), and the brand colour ignored. **Suggested fix:** Render each email once for a business in another zone (`America/Toronto`: expect "11:00 a.m. EDT" in the subject and the text). In the notification test expect the `tel:` href twice (link and button) and the brand colour. **Resolution:** Fixed 2026-10-02: each email is also rendered for a Toronto business and must read 11:00 a.m. EDT in its subject, HTML and text; the notification test expects the tel: link twice (line and button) and the brand colour. All four mutations above now fail a test. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): a hard-coded Edmonton zone in the confirmation, and the server's own zone in the notification, each fail the Toronto tests.

6/F-111 [P3] closed - The confirmation's plain-text twin never names the business when it has a logo and no phone

File: backend/emails/email-layout.tsx:46-67 (also backend/emails/booking-confirmation.tsx:36-45, 68-83; backend/emails/booking-notification.tsx:85-101) **Found:** 2026-10-02 by independent review of step 6.5 (scope: 67aaa74..fbaf9d3; lenses: quality, security, performance, tests) **Why it matters:** React Email's plain-text render skips images, so the logo, whose alt text is the business's name in the HTML, leaves nothing in the text. The name then appears only in the "Call <business>" button (only with a phone) or the footer (only without a website). Probed: a business with a logo, a website and no phone gets a text body

that never says who the booking is with. Smaller, same twin: single line breaks in the customer's words, kept in the HTML by `pre-line`, are joined into one line in the notification's text ("Two storeys, stucco. South side peeling."). The subject and the From name still carry the business, hence P3. **Suggested fix:** Name the business once in the body whatever it has (for example "Here are the details of your booking with Primo Painters."), and add a test with a logo and no phone that expects the name in the text. The line-break loss can be accepted or the fixture given a single line break. **Resolution:** Fixed 2026-10-02: the confirmation's opening line names the business ("Primo Painters has you booked."), tested for a business with a logo and no phone. The customer's single line breaks are now `<br />` instead of `pre-line`, so they survive in the plain-text twin too, tested. Both changes reverted fail a test. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): dropping the business's name from the confirmation's opening line fails the logo-and-no-phone test.

6/F-112 [P3] closed - Step numbers in two code comments

File: backend/emails/booking-email-facts-type.ts:1-2, backend/emails/booking-notification.tsx:3 **Found:** 2026-10-02 by independent review of step 6.5 (scope: 67aaa74..fbaf9d3; lenses: quality, security, performance, tests) **Why it matters:** "Step 6.6 reads it from the database" and "Reply-to is the customer (step 6.6)". The coding standards' Comments section rules out step numbers in code comments (they belong in the build log); F-106 was the same kind of slip. **Suggested fix:** "Read from the database inside the booking's own business" and "Reply-to is the customer, set where the email is sent". **Resolution:** Fixed 2026-10-02: both comments reworded without step numbers. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): neither file carries a step number in a comment; every added line of the delta was searched. Finding numbers left in three other comments are F-116, a separate entry.

6/F-113 [P2] closed - No test drives a failure out of the not-awaited send, so the guard against crashing Node is unproved

File: backend/lib/booking/booking-confirmation-emails.ts:218-223 (the paths: backend/lib/email/send-booking-emails.ts:67-72, backend/lib/email/find-business-email-details.ts:176-178) **Found:** 2026-10-02 by independent review of step 6.6 (scope: 3bee2a3..b434c85; lenses: quality, security, performance, tests) **Why it matters:** `sendBookingEmails` rejects when the saved key cannot be unlocked (the token key changed, or the credentials are bound to another business), when the database fails, or when the booking or business is gone. Only the `.catch` in `start` stands between that rejection and an unhandled rejection, which stops the API process. Mutation: deleting that `.catch` leaves all 9 tests in `send-booking-emails.test.ts` green, because every failure the tests drive (Resend refusing) is caught inside the per-email `try`. The code is right today; nothing keeps it right. The Google event's twin (`booking-event-writes.ts`) has the same gap. **Suggested fix:** One test: a business whose `email_sending_key` row holds credentials locked for another business id (or garbage), booked through `bookTime`; expect the booking saved, no Resend call, no `email_sent` entry, and one `console.warn` line with the booking id and none of Jane's details. Vitest fails the run on an unhandled rejection, so removing the `.catch` would

then fail it. **Resolution:** Fixed 2026-10-02: a new test books through bookTime with a key locked for another business, so the send fails before Resend; it expects the booking kept, no Resend call, no entry and one warning with the booking id and none of Jane's details. Removing the .catch now fails it (Vitest fails on the unhandled rejection). The Google event's twin in booking-event-writes.ts is feature 5's code and keeps its gap; noted for /complete. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): removing the .catch in booking-confirmation-emails.ts fails the unreadable-key test on the unhandled rejection.

6/F-114 [P3] closed - The booking read's two guards have no test: a cancelled booking and another business's id

File: backend/lib/email/send-booking-emails.ts:65, 69 **Found:** 2026-10-02 by independent review of step 6.6 (scope: 3bee2a3..b434c85; lenses: quality, security, performance, tests) **Why it matters:** Mutations: removing `if (row.status !== "confirmed") return [];` and dropping `eq(booking.organizationId, organizationId)` from the `where` each leave every test green. Neither is reachable wrongly today (the only caller passes the booking it just saved, in its own business), but feature 8's retries will call this with older bookings, some cancelled by feature 7, and the tenant rule is the one the brief and 6.2 hold every read to. **Suggested fix:** Two short direct calls in the same file: a booking set to `cancelled` returns `[]` with no Resend call; `sendBookingEmails(otherBusiness, bookingId)` rejects with "no such booking in this business" and sends nothing. **Resolution:** Fixed 2026-10-02: two direct calls, a cancelled booking returning `[]` with nothing sent, and another business's booking id rejected with nothing sent. Removing the status check or the business filter now fails a test each. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): removing the status check, and removing the business filter from the booking read, each fail a test.

6/F-115 [P3] closed - book-time.ts comments: the file header omits the emails, and a finding number stays in a rewritten comment

File: backend/lib/booking/book-time.ts:1-6, 330-331 **Found:** 2026-10-02 by independent review of step 6.6 (scope: 3bee2a3..b434c85; lenses: quality, security, performance, tests) **Why it matters:** The header still says what happens after the save is "the event starts going into the booked person's Google"; the emails now start there too, so the one path every booking takes is described half. The comment at 330-331 was rewritten in this step and kept "(decision 6, F-93)"; the coding standards' Comments section rules out finding numbers in code comments (F-112 was the same kind of slip). **Suggested fix:** Header: "...then the event starts going into the booked person's Google and the emails go out, without the answer waiting for either." At 331: drop "F-93" and keep the rule in words ("a failure keeps the booking"). **Resolution:** Fixed 2026-10-02: book-time.ts's header names the emails beside the Google event, and the comment no longer carries F-93. Closed 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests): book-time.ts's header names the emails beside the Google event, and the comment after the save no longer carries a finding number.

Independent review

Status: passed **Target commit:** 8858d373c582f189edf4e0d5a01da0c322a2453f **Base commit:** 7dc0721f12f297ed6ec9a776ddb31265e82574fd **Base ref:** main **Spec hash:** f3c4d89966e1cd8a66294aedcf8dc6054d797c000196c2ec38b4e9eee6f2d379 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-03T06:15:05Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-03T06:25:15Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`7dc0721f12f297ed6ec9a776ddb31265e82574fd..8858d373c582f189edf4e0d5a01da0c322a2453f` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (12 files, 103 tests)
- `npm run test --workspace=backend` : pass (48 files, 450 tests; run twice, before and after the mutations)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass
- Verify command: unavailable (none declared)

Evidence

- Preflight: HEAD equals the target, `git merge-base main HEAD` equals the base, the spec's SHA-256 matches; the only paths differing from the target are review.md, findings.md and the untracked blueprint/ai-voice-proposal.md (outside the target).
- Whole delta read (78 files, 14 commits): the seam, settings and login codes; migration 0016, the locked key table and the reader; the card's routes, save path and frontend card; the 3b form; the invite; the two templates; the not-awaited send from bookTime and the recipients function.

- End to end: a public booking saves, then bookingConfirmationEmails.start runs outside the transaction and only on the new-booking path (alreadyBooked and the request-key race return before it); the booking and business are read inside one business; the business's own key is unlocked bound to its id; idempotency keys are booking-confirmation/<id> and booking-notification/<id>; the timeline payload holds bookingId, kind and resendId only.
- Decisions 1 to 11 checked against the code: not awaited (1), one key per email and new bookings only (2), no address or content in payloads or logs (3), no sender, notify address, key or zone means nothing sent and one line logged (4), UTC times with no VTIMEZONE (5), key write-only in its own table, never returned by GET, PUT, admin or Better Auth's organization routes (6), login codes from LOGIN_EMAIL_FROM with the agency key, production refuses to start without both (7), notification to notifyEmail, defaulting to the owner's email at setup (8), the card re-tests with the saved key on every save (9), source manual skips the notification (10), no worker email (11).
- 30 mutations, each restored with git checkout. 29 caught: the await put back on login codes; a code logged; quota and wrong-key branches; the sender refusal code; plan checks; permission check; the UID domain; fold limit and UTF-16 iteration; the control-character strip; hard-coded and server zones in both templates; the business name in the opener; the .catch on the background send; the status and tenant guards on the booking read; recipients for manual bookings; reply-to; the reader's, state's and save's tenant filters; the idempotency option; the production settings check; re-testing a saved key; the timeout check; the raw code in the login key; the notify default at setup; the phone's plain message. 1 survived: removing the `status === 422` half of the sender-refusal condition (noted under F-100).

Findings

- F-116 [P3] open: finding numbers in three code comments added by this feature
- Closed this pass: F-97, F-98, F-100, F-102, F-103, F-104, F-105, F-106, F-107, F-108, F-109, F-110, F-111, F-112, F-113, F-114, F-115
- No P0 or P1 open or fixed

Remaining risk

- No Verify command and no GitHub check are declared, so there is no single command proving the whole repo.
- No real email was sent: the Resend API (a send-only key's limits, idempotency, Outlook seeing a meeting) was checked only against a faked fetch.
- Browser flows (the Email sending card, the 3b form) were not driven in this review; Check was not required, so the hand checks recorded in the build log were relied on.
- The Node on Railway must carry time zone data from 2026c or later (Alberta's permanent daylight time), or Primo's winter times read an hour off; unverified here.

- Production behaviour depends on NODE_ENV=production on Railway: the start-up refusal, sendEmail's no-key refusal and the Resend SDK's silence on errors all key on it.
- Until feature 8, an API restart between a booking's save and its sends loses those emails; until feature 9's rate limits, the public form can send a confirmation with a typed name to any address, bounded by the business's free times.
- Removing the 422 half of the sender-refusal condition leaves every test green (see F-100's closure).
- blueprint/ai-voice-proposal.md is an untracked note of Frank's, outside the reviewed target.